

Competitive Programming

Section 4: Prefix sums and difference arrays

Competitive Programming Course

2026-07-04

Table of contents

1	Why prefix sums matter	3
2	The range sum formula	4
3	Inclusive versus half-open intervals	5
4	Worked example: range sums	6
5	Difference arrays	7
5.1	Why this works	7
6	Worked example: several updates	8
7	Prefix sums as coordinates	9
8	Worked example: subarray sum count	10
9	Integer sizes	11
10	Proof habits	12
11	Common mistakes	13
12	Before you submit	14
13	Exercises	15

1 Why prefix sums matter

Prefix sums are the first data structure that often turns a problem from too slow to immediate. The idea is simple:

store cumulative information so a range can be answered by subtracting two stored values.

For an array:

```
a = 1 2 3 4 5
```

the prefix sums are:

```
prefix[0] = 0
prefix[1] = 1
prefix[2] = 3
prefix[3] = 6
prefix[4] = 10
prefix[5] = 15
```

Here `prefix[i]` means the sum of the first `i` elements. The extra zero at the beginning is deliberate. It makes ranges that start at the first element use the same formula as every other range.

2 The range sum formula

For a 1-indexed inclusive query $[l, r]$, the answer is:

$$prefix[r] - prefix[l - 1].$$

Why? `prefix[r]` contains everything from position 1 through `r`. `prefix[l - 1]` contains everything before `l`. Subtracting removes the part before the query.

```
array:      1   2   3   4   5
index:      1   2   3   4   5
query:           [ 2..4 ]

prefix[4] = 1 + 2 + 3 + 4
prefix[1] = 1
difference = 2 + 3 + 4
```

The most common prefix-sum bugs are not mathematical. They are indexing bugs.

3 Inclusive versus half-open intervals

Competitive programming statements often use 1-indexed inclusive intervals:

```
l <= i <= r
```

Many programming languages naturally use 0-indexed half-open slices:

```
l <= i < r
```

Both conventions are valid, but mixing them is dangerous.

This course uses this standard for prefix arrays:

```
prefix[0] = 0  
prefix[i] = a[1] + ... + a[i]
```

Then 1-indexed inclusive `[l, r]` becomes:

```
prefix[r] - prefix[l - 1]
```

In Python slicing terms, if `values` is 0-indexed:

```
sum(values[l - 1:r])
```

matches the same query.

4 Worked example: range sums

Array:

-5 10 -2

Prefix:

i	prefix[i]
0	0
1	-5
2	5
3	3

Queries:

query	formula	answer
[1, 2]	$\text{prefix}[2] - \text{prefix}[0]$	5
[2, 3]	$\text{prefix}[3] - \text{prefix}[1]$	8
[1, 3]	$\text{prefix}[3] - \text{prefix}[0]$	3

5 Difference arrays

Prefix sums answer many range queries on a fixed array. Difference arrays do the opposite: they apply many range updates efficiently.

Suppose we start from zeros and want to add x to every index in $[1, r]$. Instead of touching every position, mark only where the change starts and where it stops:

```
diff[l] += x
diff[r + 1] -= x
```

After all updates, take a running sum of `diff` to reconstruct the final array.

5.1 Why this works

Adding x at l means every later running sum includes x . Subtracting x at $r + 1$ means positions after r no longer include it.

For $n = 5$, update $[2, 4] += 7$:

diff:	0	+7	0	0	-7	0
index:	1	2	3	4	5	6
scan:	0	7	7	7	0	

Positions 2, 3, and 4 receive 7; positions outside do not.

6 Worked example: several updates

Start with five zeros:

```
0 0 0 0 0
```

Updates:

```
[1, 3] += 10  
[2, 5] += 1  
[4, 4] += -5
```

Difference marks:

update	diff change
[1,3] += 10	diff[1] += 10, diff[4] -= 10
[2,5] += 1	diff[2] += 1, diff[6] -= 1
[4,4] += -5	diff[4] += -5, diff[5] -= -5

Reconstructing gives:

```
10 11 11 -4 1
```

7 Prefix sums as coordinates

Prefix sums can represent subarray sums. Let:

```
prefix[i] = a1 + a2 + ... + ai
```

Then the sum of subarray $l..r$ is:

$$prefix[r] - prefix[l - 1].$$

If we want this sum to equal k , then:

$$prefix[l - 1] = prefix[r] - k.$$

So when scanning from left to right, the number of subarrays ending at the current position is the number of previous prefix sums equal to `current_prefix - k`.

This is the same complement idea from pair counting, but applied to prefix sums instead of array values.

8 Worked example: subarray sum count

Array:

```
1 2 3 -2 1
```

Target:

```
3
```

Prefix values:

```
0, 1, 3, 6, 4, 5
```

The valid subarrays are:

```
1 2
3
2 3 -2
```

There are 3, not 4. This is exactly the kind of case where a handwritten sample can be wrong unless it is checked by a slow oracle.

9 Integer sizes

Prefix sums and range-update results can exceed 32-bit integers.

If $n = 200000$ and every value is 10^9 , a prefix sum may be:

$$2 \cdot 10^5 \cdot 10^9 = 2 \cdot 10^{14}.$$

Use `long long` in C++ for sums, differences, and answers.

For subarray counts, the number of subarrays can be:

$$\binom{n+1}{2},$$

which is also too large for 32-bit `int`.

10 Proof habits

Prefix-sum proofs usually say:

`prefix[i]` equals the sum of the first `i` elements.

Then the range formula follows by subtraction.

Difference-array proofs usually say:

the running sum at position `i` includes exactly the updates whose range has started but not yet ended.

Prefix-frequency proofs usually say:

before processing the current position, the map counts all previous prefix sums.

11 Common mistakes

- Forgetting `prefix[0] = 0`.
- Using `prefix[r] - prefix[1]` for a 1-indexed inclusive query.
- Allocating `diff` with only `n + 1` cells and writing `diff[r + 1]`.
- Forgetting that `r = n` still needs a safe `r + 1` marker.
- Counting only positive arrays; prefix-sum frequency works with negatives.
- Forgetting `freq[0] = 1` for subarrays starting at the first element.
- Using 32-bit `int` for sums or counts.

12 Before you submit

For range sums:

1. Is the input interval 0-indexed or 1-indexed?
2. Is it inclusive or half-open?
3. Does the formula handle $l = r$?

For difference arrays:

1. Is `diff` large enough for $r + 1$?
2. Did you add at l and subtract after r ?
3. Are negative updates allowed?

For subarray counting:

1. Did you initialize the zero prefix count?
2. Do you count before inserting the current prefix?
3. Are negative values allowed?

13 Exercises

The local exercises cover three core patterns:

1. `a_range_sum_queries`: static range sums.
2. `b_range_add_final_array`: range updates and reconstruction.
3. `c_subarray_sum_count`: prefix sums plus frequency maps.

Run:

```
python3 sections/04_prefix_difference/check.py
```